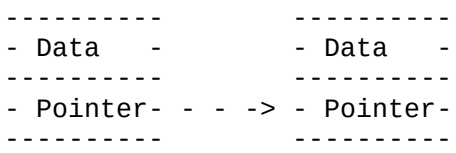
## Lesson 15: Singly linked lists

Linked lists are a way to store data with structures so that the programmer can automatically create a new place to store data whenever necessary. Specifically, the programmer writes a struct or class definition that contains variables holding information about something, and then has a pointer to a struct of its type. Each of these individual struct or classes in the list is commonly known as a node.

Think of it like a train. The programmer always stores the first node of the list. This would be the engine of the train. The pointer is the connector between cars of the train. Every time the train adds a car, it uses the connectors to add a new car. This is like a programmer using the keyword new to create a pointer to a new struct or class.

In memory it is often described as looking like this:




The representation isn't completely accurate, but it will suffice for our purposes. Each of the big blocks is a struct (or class) that has a pointer to another one. Remember that the pointer only stores the memory location of something, it is not that thing, so the arrow goes to the next one. At the end, there is nothing for the pointer to point to, so it does not point to anything, it should be a null pointer or a dummy node to prevent it from accidentally pointing to a totally arbitrary and random location in memory (which is very bad).

So far we know what the node struct should look like:

```
struct node {
  int x;
  node *next;
};

int main()
{
  node *root;      // This will be the unchanging first node

  root = new node; // Now root points to a node struct
  root->next = 0;  // The node root points to has its next pointer
                   //  set equal to a null pointer
  root->x = 5;     // By using the -> operator, you can modify the node
                   //  a pointer (root in this case) points to.
}
```

This so far is not very useful for doing anything. It is necessary to understand how to traverse (go through) the linked list before going further.

Think back to the train. Lets imagine a conductor who can only enter the train through the engine, and can walk through the train down the line as long as the connector connects to another car. This is how the program will traverse the linked list. The conductor will be a pointer to node, and it will first point to root, and then, if the root's pointer to the next node is pointing to something, the "conductor" (not a

technical term) will be set to point to the next node. In this fashion, the list can be traversed. Now, as long as there is a pointer to something, the traversal will continue. Once it reaches a null pointer (or dummy node), meaning there are no more nodes (train cars) then it will be at the end of the list, and a new node can subsequently be added if so desired.

Here's what that looks like:

```
struct node {
  int x;
  node *next;
};

int main()
{
  node *root;       // This won't change, or we would lose the list in memory
  node *conductor;  // This will point to each node as it traverses the list

  root = new node;  // Sets it to actually point to something
  root->next = 0;   //  Otherwise it would not work well
  root->x = 12;
  conductor = root; // The conductor points to the first node
  if ( conductor != 0 ) {
    while ( conductor->next != 0)
      conductor = conductor->next;
  }
  conductor->next = new node;  // Creates a node at the end of the list
  conductor = conductor->next; // Points to that node
  conductor->next = 0;         // Prevents it from going any further
  conductor->x = 42;
}
```

That is the basic code for traversing a list. The if statement ensures that there is something to begin with (a first node). In the example it will always be so, but if it was changed, it might not be true. If the if statement is true, then it is okay to try and access the node pointed to by conductor. The while loop will continue as long as there is another pointer in the next. The conductor simply moves along. It changes what it points to by getting the address of conductor->next.

Finally, the code at the end can be used to add a new node to the end. Once the while loop as finished, the conductor will point to the last node in the array. (Remember the conductor of the train will move on until there is nothing to move on to? It works the same way in the while loop.) Therefore, conductor->next is set to null, so it is okay to allocate a new area of memory for it to point to. Then the conductor traverses one more element (like a train conductor moving on the the newly added car) and makes sure that it has its pointer to next set to 0 so that the list has an end. The 0 functions like a period, it means there is no more beyond. Finally, the new node has its x value set. (It can be set through user input. I simply wrote in the '=42' as an example.)

To print a linked list, the traversal function is almost the same. It is necessary to ensure that the last element is printed after the while loop terminates.

For example:

```
conductor = root;
if ( conductor != 0 ) { //Makes sure there is a place to start
```

```
  while ( conductor->next != 0 ) {
    cout<< conductor->x;
    conductor = conductor->next;
  }
  cout<< conductor->x;
}
```

The final output is necessary because the while loop will not run once it reaches the last node, but it will still be necessary to output the contents of the next node. Consequently, the last output deals with this. Because we have a pointer to the beginning of the list (root), we can avoid this redundancy by allowing the conductor to walk off of the back of the train. Bad for the conductor (if it were a real person), but the code is simpler as it also allows us to remove the initial check for null (if root is null, then conductor will be immediately set to null and the loop will never begin):

```
conductor = root;
while ( conductor != NULL ) {
  cout<< conductor->x;
  conductor = conductor->next;
}
```